ECE532 Project

FPGA SPICE Circuit Simulator

David Montes de Oca

Sahil Amin

Ibrahim Sweidan

Andrew Radke

April 7th, 2026

TABLE OF CONTENTS

1. OVERVIEW **2**
1.1 Background 2
1.2 Goals 3
1.3 IP Descriptions 4
2. PROJECT OUTCOMES **4**
3. PROJECT SCHEDULE **7**
3.1 Planned Milestones 7
3.2 Actual Milestones 7
3.3 Comparison and Discussion 9
4. SYSTEM DESIGN OVERVIEW **10**
4.1 Utilization and Timing 10
4.1.1 Resource Utilization 10
4.1.2 Timing Performance 10
4.2 IP Block Descriptions 11
4.2.1 Parser Software 11
4.2.2 VGA Display Software 12
4.2.3 AXI-Slave Wrapper 13
4.2.4 LU System Top IP (lu_system_top) 13
4.2.5 LU Decomposition IP (lu_decomp) 15
4.2.6 LU Solver IP (lu_solver) 16
4.3 Verification Process 17
5. DESIGN TREE AND VIDEO **17**
6. TIPS AND TRICKS **18**
7. REFERENCES **18**

1. OVERVIEW

1.1 Background

Existing circuit simulators such as the Simulation Program with Integrated Circuit Emphasis (SPICE) provide powerful analysis capabilities and mature user interfaces, but rely primarily on software execution. In contrast, FPGA-based approaches can exploit parallel hardware execution and tight integration with input and display peripherals to potentially achieve greater performance (especially when the number of nodes in the circuit scales), however at the cost of increased design complexity and reduced portability.

The core of the algorithm is based on Modified Nodal Analysis (MNA). This allows the circuit to be expressed as a system of linear equations in the form $Ax = b$, where A captures the circuit topology and element parameters, x contains the unknown node voltages and source currents, and b represents the independent voltage and current sources in the circuit.

LU decomposition is commonly used to solve this system of equations [1]. This is the factorization of any invertible matrix into a lower and upper triangular matrix, as shown below.

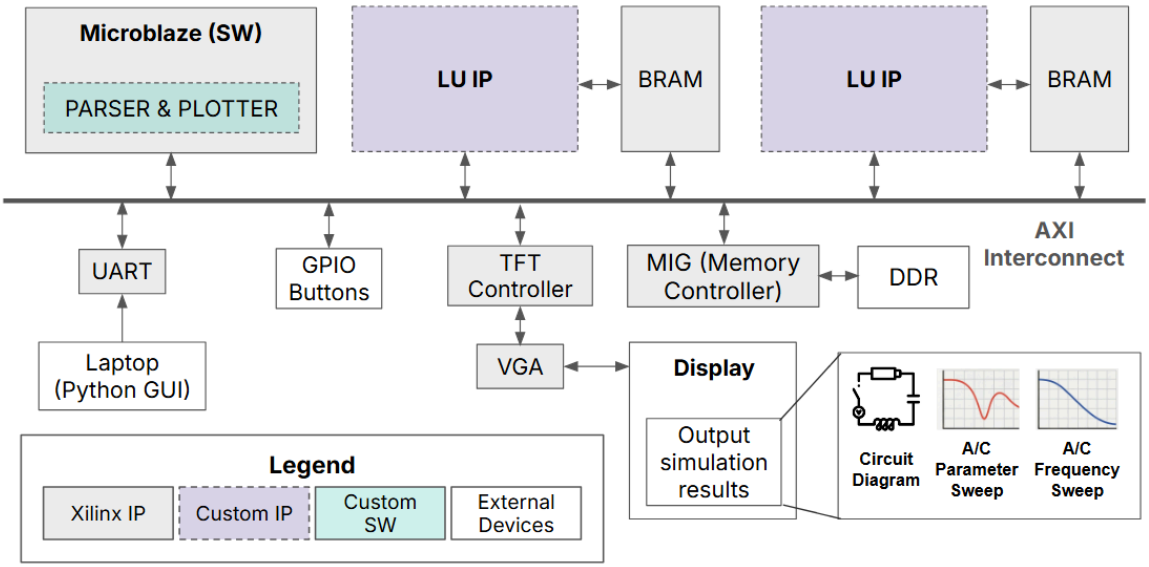
$$\begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ A_{31} & A_{32} & A_{33} & A_{34} \\ A_{41} & A_{42} & A_{43} & A_{44} \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 0 & 0 & 0 \\ L_{21} & 1 & 0 & 0 \\ L_{31} & L_{32} & 1 & 0 \\ L_{41} & L_{42} & L_{43} & 1 \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} & U_{13} & U_{14} \\ 0 & U_{22} & U_{23} & U_{24} \\ 0 & 0 & U_{33} & U_{34} \\ 0 & 0 & 0 & U_{44} \end{bmatrix}$$

Figure 1: Standard matrix form for LU decomposition.

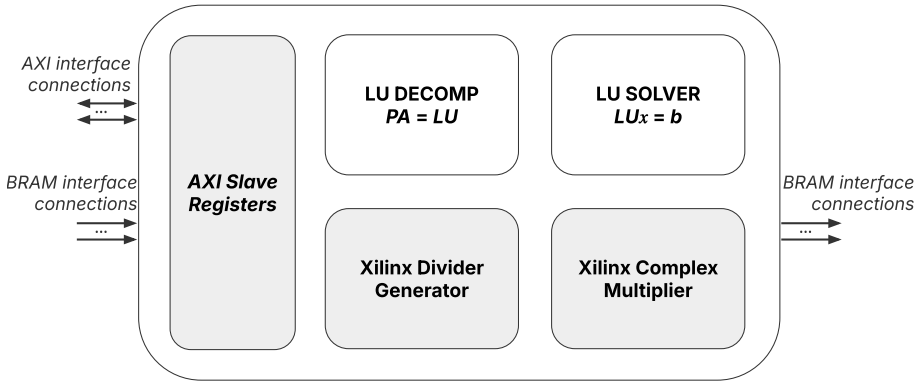
The main advantage of this is that it makes the solving of a system of linear equations defined by $Ax = b$ significantly simpler. In cases when there are multiple systems of equations in which the A stays constant but the b vector changes, the factorized L and U can be reused. Such is the case with our voltage parameter sweep. Additionally, MNA matrices are often sparse, and various LU algorithms can make use of this to achieve better performance.

1.2 Goals

The goal of this project is to demonstrate how reconfigurable hardware can be used to implement and visualize circuit simulation in a fully functional, standalone system. While high performance is a motivating objective, the minimum viable product prioritizes functional correctness and completeness of the simulation pipeline over raw throughput. We aim to design a self-contained circuit simulator on the Nexys4-DDR, in which users interact through an interface displayed via VGA.



a)



b)

Figure 2: a) Final block diagram for SPICE Circuit Simulator. b) Block diagram of custom IP top level

1.3 IP Descriptions

TABLE I: Descriptions of SPICE Circuit Simulator components.

Component	Description
MicroBlaze	Xilinx soft processor core for parsing and writing VGA screen to TFT controller.
UART	AXI core for serial communication for transmitting and receiving data.
GPIO Buttons	AXI core for reading physical push-button states.
TFT Controller	AXI core for driving VGA screen over a parallel interface.
VGA	Video output interface for generating analog RGB signals to a monitor.
MIG	Memory Interface Generator; configures and manages external memory controllers.
DDR	Double data rate SDRAM for high-bandwidth external memory storage.
LU IP	Custom IP block for solving linear systems of equations.
BRAM	On-chip block RAM embedded within the FPGA for fast, local data storage.

2. PROJECT OUTCOMES

With regards to the proposed design, we have successfully completed the majority of our initial project objectives. The simulator currently supports a variety of circuit components and multiple simulation types. A summary of the functionality of our simulator is provided in TABLE II.

TABLE II: Initial Project Milestones.

Objective	Type	Implemented
Resistors, Inductors & Capacitors	Component	Yes
Voltage & Current Sources	Component	Yes
DC Operating Point	Simulation	No
AC Steady State	Simulation	Yes
AC Sweep	Simulation	Yes
Parameter Sweep	Simulation	Yes
Transient	Simulation	No

While the project met the performance specifications for the supported simulation types, the solver is constrained by a static matrix size, meaning the IP is hard-coded to handle one specific NxN matrix size at a time. If an inputted circuit results in a different matrix size, the solver will error because it cannot dynamically reconfigure at runtime. Additionally, we identified a significant performance bottleneck in the MicroBlaze parser. Currently, the system spends over 98% of its execution time on processing SPICE strings into an MNA matrix. In other words, at our current matrix sizes, the software overhead dominates the total execution time. This bottleneck may eventually be relieved as the MNA process has time complexity O(n), whereas the LU solver is $O(n^3)$. This means that as we scale to larger circuits, we expect the time complexity of the solver to eventually outweigh the cost of parsing.

If this project were to be extended, we recommend several future steps:

1) **Dynamic Matrix Sizing and Clock Domain Splitting:** The most immediate step is to modify the IP to support variable matrix sizes. This would allow the simulator to handle any arbitrary circuit without requiring resynthesis. Additionally, the critical timing path is in the custom IP and hence controls the Fmax. Since the MicroBlaze and the IP are within the same clock domain, the MicroBlaze is not being run at its maximum frequency. To improve speed, the design should be separated into multiple clock domains, allowing the MicroBlaze to run faster and speeding up the parsing stage.

2) Symbolic MNA for AC Sweeps: To further mitigate the software bottleneck during AC sweeps, symbolic MNA could be implemented. Instead of using the component values, the variables themselves can be used during the initial parse, making it so that the system would only need to perform the matrix construction once. Subsequent simulation points would only require a numerical substitution, which would eliminate the repetitive parsing that the MicroBlaze does during sweeps. This has the potential to increase the speed of subsequent solves by over 50x. However, this algorithm is much more complex and would substantially increase the overhead for a single matrix construction.

3) Sparse Matrix Solver: In order to keep increasing to larger circuits, transitioning the hardware solver to handle sparse matrices would be essential. Because MNA circuit matrices are inherently sparse, a regular solver is inefficient with resources and time. Exploiting sparsity would allow the system to handle significantly larger circuits by reducing the memory, number of operations and FPGA resources required.

At the beginning of the project, we underestimated how much of a bottleneck the MicroBlaze parser would be on the system. If we were to redo the project, one thing we would do differently is explore different architectures to exploit hardware parallelism in the algorithm. By splitting the software component into multiple parts, some could be mapped to hardware, alleviating the massive overhead the MicroBlaze has and potentially speeding up the system.

3. PROJECT SCHEDULE

This section compares our original project schedule with final outcomes.

3.1 Planned Milestones

TABLE III summarizes the initial milestone structure the team decided upon for the project proposal.

TABLE III: Initial Project Milestones

Milestone	Objectives	Deliverables
1	MNA matrix formation + IP Research	Half of the team focus on MNA, the other half research implementations for LU Decomposition and Solver IPs.
2	Input Integration (PS/2 Keyboard, UART + Python script)	Half of the team focuses on each input method's integration, the other half begins development of each IP.
3	LU Decomposition IP for AC Analysis	Complete LU Decomposition IP with Complex Number support for A/C Sweeps.
4	LU Solver IP for AC Analysis + System Integration	Complete LU Solver IP with Complex Number support for AC sweeps, and integrate end-to-end on FPGA.
5	VGA Output with Plotting + Circuit Drawing	Take outputs from FPGA to produce meaningful plots for various simulation types (transient, AC sweep, DC parameter sweep).
6	SD Card Integration	Integrate SD Card to save and load circuit netlists for resimulation.

3.2 Actual Milestones

TABLE IV outlines a summary of the final project milestones achieved, with a more detailed account of each member's contributions towards a given milestone.

TABLE IV: Final Project Milestones and Accomplishments by each group member.

Milestone	Objectives	Accomplishments
1	MNA matrix formation + IP Research	● David and Andrew focused on MNA Parsing. ● Ibrahim and Sahil focused on research for LU Decomposition and Solver IPs.
2	Input Integration (PS/2 Keyboard, UART + Python script)	● Achieved PS/2 keyboard integration (David). ● Achieved UART integration & Parser SW (Andrew). ● Developed fixed-point, real-valued input implementation for LU Decomposition (Ibrahim) and LU Solver IPs (Sahil).
3	System Integration Preparation for Demo	● Achieved VGA integration of text output (David). ● Finalized input flow from laptop to Microblaze via Python script (Andrew). ● Packaged custom IP as AXI4 Slave (Ibrahim). ● Verified end-to-end functionality via randomized testbench (TB) with AXI-VIP (Sahil).
-	Resistive Network Midproject Demo	● Achieved real-valued (resistor network) circuit solving for one AC operating point.
4	Complex Number Integration Preparation	● Started circuit drawing feature on VGA (David). ● Started Parser complex number support (Andrew). ● Achieved Complex Multiplier Xilinx IP integration into Custom IP (Ibrahim, Sahil) with TB verification.
5	Complex Number Integration Completed (in simulation)	● Added basic plotting functionality on VGA (David). ● Finalized Parser complex number support (Andrew). ● Achieved complex divide for custom IPs (Ibrahim). ● Verified end-to-end functionality via randomized TB (Sahil).
6	Finalize Circuit Simulator System Integration (on FPGA)	● Achieved circuit drawing, dynamic plot generation based on simulation type & GPIO button support (David). ● Finalized SW logic for all simulation types (Andrew). ● Resolved custom IP integration issues for complex numbers (Ibrahim, Sahil). ● Optimized IP by further staging arithmetic, and added re-use LU matrix functionality (Sahil).

3.3 Comparison and Discussion

The first two milestones aligned with the original plan, however subsequent milestones diverged. The main changes and their rationale are explained below.

1) Complex Number Support: Unforeseen technical challenges pushed our goal for complex number support further in the project timeline. Thus, the team opted to split our IP development into two stages: achieving end-to-end system integration for AC simulations with a purely resistive network and then incorporating capacitors and inductor support. This enabled an iterative development approach, delivering a functional real-valued system first for the midproject demonstration, followed by the extension of complex-valued processing for the final demonstration.

2) Reduction of Supported Peripherals: The team intended to integrate the PS/2 Keyboard for VGA interaction and an SD card for saving and loading circuit netlists to increase project complexity. However, these components were deemed non-essential to the core operation of the system. Their removal afforded more time towards supporting a greater number of circuit elements and simulation types, as well as for refining our hardware and software to support larger matrices. The keyboard was replaced with the GPIO buttons, which provided equivalent functionality with reduced implementation complexity, while the save and load feature was omitted entirely.

3) Reduction of Supported Simulation Types: The team planned to support transient simulations. However, due to time constraints and a significant increase in parser software complexity required to accommodate this feature, it was left unsupported for the final demonstration. We opted to prioritize voltage sweep simulation support, which showcases the performance benefit of LU decomposition through matrix reuse.

Overall, while there were deviations from the original schedule, they were driven by practical implementation challenges. These measures helped the team focus on the key requirements of the system and a more incremental development process, which ensured a successful end-to-end system integration within the project timeline.

4. SYSTEM DESIGN OVERVIEW

4.1 Utilization and Timing

This section presents the post-implementation resource utilization and timing results for the design. All results are reported after place and route to reflect real world performance.

4.1.1 Resource Utilization

Resource utilization is a design's ability to fit within the hardware provided. The table below provides the utilization of one LU decomposition and solver IP core, along with the usage of the entire project and their percentages of the device's capacity.

TABLE V: Resource utilization of one IP core and the entire IP project, along with total utilization.

Component	**Slice LUTs**	**Slice Registers**	**BRAM Tiles**	**DSP**
axi_lu_decomp_solver _v1_0_S00_AXI	128	140	0	0
cmpy_0__1	95	286	0	16
cmpy_0	95	256	0	16
div_gen_q1616__1	1760	4861	0	0
div_gen_q1616	1758	4861	0	0
lu_decomp	10327	8979	0	32
lu_solver	2828	5303	0	24
Total Project Usage	52534	69645	16.5	181
Utilization Ratio	0.83	0.12	0.12	0.75

4.1.2 Timing Performance

This report tests the design's ability to meet setup and hold time requirements across all paths. The worst negative slack (WNS) and worst hold slack (WHS) indicate the timing margin. Given our 16.667 ns clock period, the post-implementation timing results were satisfactory with the following margins:

- WNS: 0.334 ns
- WHS: 0.014 ns

Using the below formula, we can also calculate the theoretical maximum frequency the design can be clocked at without time violations, F_{max}.

$$F_{max} = \frac{1}{Clock\ Period - WNS} = \frac{1}{16.667 - 0.334} \approx 61.23\ MHz$$

With the final clock period corresponding to a 60 MHz clock speed, this timing summary shows that the design could run at a faster clock than the chosen baseline.

It is also worth noting that the logic delay along the critical path is significantly lower than the net (routing) delay. This imbalance is due to the use of distributed RAM rather than BRAM for internal buffers. While distributed RAM avoids the one-cycle read/write latency penalty inherent to BRAM, it comes at the cost of utilizing a larger portion of the FPGA's LUT fabric. Transitioning to BRAM would reintroduce that one-cycle access penalty, but could unlock substantial performance gains at the cost of additional pipeline adjustments, particularly for scaling to larger matrix sizes where the capacity and dedicated routing of BRAM become increasingly advantageous.

4.2 IP Block Descriptions

The following section will give a detailed description of each hardware IP block developed in the project, along with the C code run on the MicroBlaze soft processor. The order follows the system flow, with the software functioning as the upstream and downstream graphical user interface (GUI), and the hardware IP in between functioning as the main processing blocks.

4.2.1 Parser Software

The SPICE parser is done entirely on the MicroBlaze in C. The parser has the following steps: receive SPICE file from laptop, parse string into netlist, create MNA matrix from the netlist, put matrix into BRAM and start hardware IP, and receive results from hardware IPs.

The first step is to receive the matrix from the laptop. This is done through a UART channel from the laptop, and the MicroBlaze polls this channel until it receives the string. Once it has this string, it saves it into local BRAM.

Once it has the string it can start parsing it into the netlist. Firstly, it takes the header section of the code and determines the type of simulation and any simulation settings. The parser then goes line by line through the SPICE code and saves each element in a struct. This struct contains all the information related to the element, including its type, value and nodes it is between.

The parser then uses the list of elements to create the MNA matrix. It first creates a 0 matrix of the proper size and then starts filling it up with values. Element by element the parser will calculate the impedance and "stamp" or add the value into the correct place in the matrix [2].

Once the matrix is created, it will save it to BRAM for the LU decomposition IP to access and trigger the IP to start by writing to its AXI-slave registers. It will then poll the LU decomposition IPs flags until it is finished and write the results to BRAM so they can be plotted on the VGA. The parser will repeat the creation of the matrix and IP interactions until all the points of a sweep are complete.

4.2.2 VGA Display Software

The VGA display is done entirely on the MicroBlaze in C and has two main sections: drawing the schematic for the parsed SPICE code, and plotting the results of the simulation.

The circuit drawing is done after the netlist is generated from the parser, so its function is called before the hardware IP starts. It starts by breaking up the circuit drawing area into a 7x4 grid with each cell being 40x40 pixels. It then parses the netlist and searches for the element type, followed by its two terminals. If one of the terminals is ground, i.e. node 0, then it draws the element shunt to ground. A list of the elements in the top row is created, keeping track of the nets their terminals are connected to, to then draw the corresponding shunt elements to ground. The simple architecture of the drawer restricts it to simple high pass and low pass filter circuits, with up to three stages of RLC series and shunt elements, and a voltage source to the left.

The plotting of the simulation is done at the end of the flow, once the result has been written to BRAM. It reads the real and imaginary values, converts them to magnitude and phase, and plots them in a standard bode plot structure, i.e. plotted against logarithmic frequency with the magnitude on top in dB and the phase on bottom in degrees. The frequency values plotted are

hard coded to allow no integer overflows or underflows with the Q16.16 number format. After that, the plotting is simple mapping to fit in the range of the plots.

After plotting, a simple event loop starts to poll the status of the buttons on the FPGA, to allow the user to traverse the points on the plot, or to signal the laptop through UART that the FPGA is ready to receive another SPICE file.

4.2.3 AXI-Slave Wrapper

We have decided to use a simple AXI-Slave interface to receive the offset address locations from Microblaze of where it has stored the relevant MNA matrices in BRAM. We have 4 registers:

1) **Status/Start Registers**
    a) Microblaze programs this high to indicate when to start LU decomposition.
    b) It also acts as a status register for debugging and event-control for Microblaze.
2) **A matrix address in BRAM**
3) **B vector address in BRAM**
4) **X output address in BRAM**

Additionally it uses a simple FSM to interface directly with the BRAM ports and reads the A and b matrix/vectors. The BRAM is configured as an 8kB true dual port, to allow the microblaze to use AXI to write into BRAM, and our IP block uses the native ports. When the LU Solver IP has completed, we write the solved unknowns into BRAM at the address specified.

4.2.4 LU System Top IP (lu_system_top)

The LU system top module integrates the custom LU decomposition and solver blocks with supporting Xilinx arithmetic IPs to perform matrix factorization and solution. This module coordinates data movement and sequences the execution of four main submodules: lu_decomp and lu_solver (discussed in *Sections 4.2.5 and 4.2.6*), as well as div_gen_q1616 (Xilinx Divider Generator v5.1) and cmpy_0 (Xilinx Complex Multiplier v6.0). There are two instantiations of each of the Xilinx IPs - one pair for lu_decomp, the other for lu_solver. Detailed below are the final configurations for each existing IP:

1) Xilinx Divider Generator v5.1 (div_gen_q1616)

This IP assumes Radix2, signed representation for inputs and outputs for division. [3]

Inputs:

- s_axis_dividend (numerator): 48-bits wide for Q32.16 representation. Original input is Q16.16 that is left shifted by 16 (number of fractional bits) to ensure correct fixed-point division operation.
- s_axis_divisor (denominator): 32-bits wide for Q16.16 representation.

Outputs:

- m_axis_dout: 80 bits wide, concatenation of {Quotient, Remainder}
    - Quotient: 48-bits wide (Q32.16). The system truncates the upper 16 bits to restore 32 bit wide output (Q16.16).
    - Remainder: 32-bits wide, not needed for the system.

2) Xilinx Complex Multiplier v6.0 (cmpy_0)

This IP performs signed two's complement fixed point arithmetic, and assumes this representation in the inputs provided. [4]

Inputs:

- s_axis_a: 64-bits wide, concatenation of {imaginary_a, real_a} with each component being Q16.16 representation.
- s_axis_b: identical configuration as above, concatenation of {imaginary_b, real_b}.

Outputs:

- m_axis_dout: 144 bits wide, concatenation of {imaginary_out, real_out}
    - imaginary_out: 65-bits wide (m_axis_dout[136:72]), system truncates 17 bits from MSB and 16 bits from LSB to obtain a 32 bit output (Q16.16).
    - real_out: 65-bits wide (m_axis_dout[64:0]), system truncates 17 bits from MSB and 16 bits from LSB to obtain a 32 bit output (Q16.16).

Both the divider and multiplier units use AXI4-Stream interfaces, but each is configured as "non-blocking". Thus, each input and output signal only uses TVALID and TDATA for handshaking and has no backpressure mechanism (TREADY assumed high). This simplified control logic, but required exact scheduling of inputs for correct operation.

While the complex multiplication is handled directly by the cmpy_0 module, complex division is not supported by a Xilinx IP directly. Thus, the lu_decomp and lu_solver modules use the following identity to perform this: $\frac{a+jb}{c+jd} = \frac{ac+bd}{c^2+d^2} + j\frac{bc-ad}{c^2+d^2}$. This is staged to split up the computation over multiple cycles, and takes advantage of the pipelined IP to overlap latencies of the real and imaginary component calculations.

4.2.5 LU Decomposition IP (lu_decomp)

We are using gaussian elimination with partial pivoting to factorize the matrix A of size N [5]. We are also encoding any row swaps using a permutation vector P. Our algorithm is as follows:

Initialize the L matrix to be the identity matrix.
Copy A matrix into U matrix
Initialize P to be $[1\ 2\ 3\ \ldots\ N]$
For each column k in matrix U from 1 to N:

1) **Choose pivot row**
    a) Scan down column k from row k to N
    b) Select the row whose entry has the largest magnitude
    c) This partial pivoting prevents division by 0 and improves numerical stability
2) **Swap rows**
    a) Swap selected pivot row into row k of U matrix
    b) Swap corresponding entries in P
    c) Swap any previously computed entries in L for columns 1 to k
3) **Compute Elimination Factors (entries of L)**
    a) For each row $i > k$:
        i) Compute $\frac{U(i,k)}{U(k,k)}$
        ii) As entries are complex numbers, division is calculated as follows: $\frac{a+jb}{c+jd} = \frac{ac+bd}{c^2+d^2} + j\frac{bc-ad}{c^2+d^2}$
            (1) This then is fed to the Xilinx divider in LU system top module

iii) Store the result in $L(i,\ k)$

4) **Perform elimination**

a) For each column $j >= k$:

i) $U(i,\ j)\ \rightarrow\ U(i,\ j)\ -\ L(i,\ k)\ *\ U(k,\ j)$

(1) The complex multiply is done using the Xilinx IP in system top

5) **Advance to the next column**

a) Increment k

b) Repeat until all columns are processed, or a pivot is 0 in which case we set the fail flag high.

4.2.6 LU Solver IP (lu_solver)

Given that MNA provides a linear system representation of a given circuit in the form $Ax\ =\ b$ and the decomposition process transforms this into $LUx\ =\ b$, solving for the unknown x is achieved as follows [6]:

1) **Solve $Ly\ =\ b$ using forward substitution**

For each row i in matrix L from 1 to N:

a) Reset sum to zero

b) For each column j in matrix L from 1 to i-1, compute: $sum\ =\ sum\ +\ L(i,j)\ \cdot\ y_j$

c) Compute current row i unknown: $y_i =\ (b_i\ -\ sum)\ /\ L(i,i)$

2) **Solve $Ux\ =\ y$ using backward substitution**

For each row i in matrix U from N to 1:

a) Reset sum to zero

b) For each column j in matrix U from i+1 to N, compute: $sum\ =\ sum\ +\ U(i,j)\ \cdot\ x_j$

c) Compute current row i unknown: $x_i =\ (y_i\ -\ sum)\ /\ U(i,i)$

Both forward and backward substitution require complex addition and multiplication for step b, and complex subtraction and division for step c. Addition and subtraction were performed combinationally. Complex multiplication and division are performed as described previously in *Section 4.2.4*.

4.3 Verification Process

To ensure correct operation individually, verification was first performed at a module level for each of the components. Once this was done, we moved onto system level testing with an AXI-VIP based testbench. This allowed us to replicate a similar input flow to our final system by writing the input matrices to the BRAM, and signaling the LU IP by writing into its AXI-slave registers. Once system outputs were written to BRAM, they were validated by recomputing Ax using the solved vector x and comparing it against the original b, allowing for a small margin of error due to fixed-point precision and error propagation.

Regarding test matrices, we started with a simple matrix that would serve as a sanity check for the system. Once our system produced correct outputs for this case, we moved to randomized inputs for the real and imaginary components. This was done for a large number of iterations to ensure confidence in the operation, and done with limits to the randomization to account for overflow of the internal registers for arithmetic.

5. DESIGN TREE AND VIDEO

The github repository (https://github.com/isweidan/FPGA-Circuit-Simulator) contains our (System)Verilog and C language code for our ECE532 Project, which involved creating an end-to-end SPICE simulator on a Xilinx FPGA.

Please refer to the system introduction video for a brief overview of the project and final achievements.

The repository is organized in the following structure:

- **IP_modules:** (System)Verilog files involved in our AXI-wrapped LU Decomposition and Solver IP, alongside testbenches used for system verification.
- **software:** Microblaze soft-processor C-code for UART input and VGA output.
- **verification:** testbench for system-level verification, as well as spice code samples used for final demonstration.
- **vivado_project:** Archived project for final packaged LU Decomposition and Solver IP.
- **docs:** PDFs of final report and presentation.

6. TIPS AND TRICKS

- Don't treat Microblaze as a black box, remember to consider stack/heap/code size and where it fits in your system, stack overflows are not flagged when the code is run.
- Read warning messages from Vivado - even though your design generates bitstream, that doesn't mean the design is correctly connected. In our case, our LU Decomposition logic was optimized away due to a multiple-drivers warning which was overlooked initially.
- Learning how to use the ILA properly can save hours of debugging.
- Make sure to read the documentation thoroughly of each Xilinx IP you are including in your design, things such as clock speed, data widths, logic signals, and reset polarity can be easily missed.
- When interfacing between hardware and software, be very conscious as to what number formatting scheme you are working in.
- If making an output GUI, draw it out first by hand to stage all the components and parametrize everything to make small pixel adjustments simple.
- Make sure to understand variations between different AXI protocols and select what is required for your application to reduce unnecessary workload.

7. REFERENCES

[1] D. Niu, Y. Tao, Z. Jin, Y. Dong, C. Wang, and C. Sun, "ISLU: Indexing-Efficient Sparse LU Factorization for Circuit Simulation on GPUs (Invited Paper)," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD 2024)*, 2025, Art. no. 23, pp. 1–9, doi: 10.1145/3676536.3695410.

[2] F. N. Najm, Circuit Simulation. Hoboken, NJ, USA: John Wiley & Sons (Wiley-IEEE Press), 2010.

[3] AMD (Xilinx), Divider Generator v5.1 LogiCORE IP Product Guide (UG958/PG151), Feb. 2021. [Online]. Available: https://docs.amd.com/r/en-US/ug958-vivado-sysgen-ref/Divider-Generator-5.1

[4] Xilinx Inc., Complex Multiplier v6.0 LogiCORE IP Product Guide (PG104), Dec. 2013. [Online]. Available:

https://www.eecg.utoronto.ca/~pc/courses/432/2014/projects/5_notesensing/doc/Datasheets/complex_multiplier.pdf

[5] University of Alberta, "LU decomposition," *Engineering at Alberta Courses*. [Online]. Available: https://engcourses-uofa.ca/books/numericalanalysis/linear-systems-of-equations/direct-methods/lu-decomposition/.

[6] D. Cherney, T. Denton, R. Thomas, and A. Waldron, *Linear Algebra*. Davis, CA, USA: University of California, Davis, 2016. [Online]. Available: https://www.math.ucdavis.edu/~linear/.